COMPLETE TECHNICAL AUDIT REPORT

Police Lines School & College, Faridpur — Reunion Management Website

Audit Date: 2026-11-11

Auditor: Senior Software Architect / Security Auditor

Scope: Full codebase, database schema, API surface, authentication, authorization, and security posture

Status: Read-only audit — no changes made

TABLE OF CONTENTS

1. [Executive Summary](#1-executive-summary)
2. [Technology Stack](#2-technology-stack)
3. [System Architecture](#3-system-architecture)
4. [Database Table Inventory](#4-database-table-inventory)
5. [Database Relationship Analysis](#5-database-relationship-analysis)
6. [Complete ER Diagram](#6-complete-er-diagram)
7. [API & Edge Function Documentation](#7-api--edge-function-documentation)
8. [Data Flow Documentation](#8-data-flow-documentation)
9. [Authentication & Authorization Analysis](#9-authentication--authorization-analysis)
10. [User-to-User Data Isolation Test Results](#10-user-to-user-data-isolation-test-results)
11. [Security Vulnerability Report](#11-security-vulnerability-report)
12. [Database Performance Analysis](#12-database-performance-analysis)
13. [Broken or Missing Connections](#13-broken-or-missing-connections)
14. [Role & Permission Matrix](#14-role--permission-matrix)
15. [Visual Architecture Diagrams](#15-visual-architecture-diagrams)
16. [Recommended Improvements & Prioritized Action Plan](#16-recommended-improvements--prioritized-action-plan)

---

1. EXECUTIVE SUMMARY

The Police Lines School & College, Faridpur Grand Reunion website is a single-page React application (SPA) deployed on Vercel, backed by Supabase (PostgreSQL + Auth + Storage + Edge Functions) and integrated with the PayStation payment gateway.

What Works '

- • Student registration (former and current) with dynamic fee calculation
- • PayStation payment gateway integration with server-side verification via Edge Function
- • QR-code ticket generation and secure ticket lookup with rate limiting
- • Multi-role admin dashboard with 12+ tabs
- • 12-module Reunion Management System (RMS/ERP)
- • Student ID Card generation (CR80 format, 54×86 mm) with batch ZIP downloads
- • Event-day scanning: gate entry/exit, food distribution, dress distribution
- • Financial ledger and staff budget allocation
- • Committee management, volunteer roster, task board, meetings, weekly reports
- • XSS protection via DOMPurify + sanitizeInput() throughout
- • CSP headers enforced via vercel.json (HTTP headers, not meta-tag only)
- • Immutable tamper-evident audit logs with HMAC-SHA256 checksums
- • Row Level Security (RLS) enabled on all Supabase tables
- • Cryptographic HMAC-SHA256 payment signature (server-side, in Edge Function)

What Is Broken or Problematic &þ

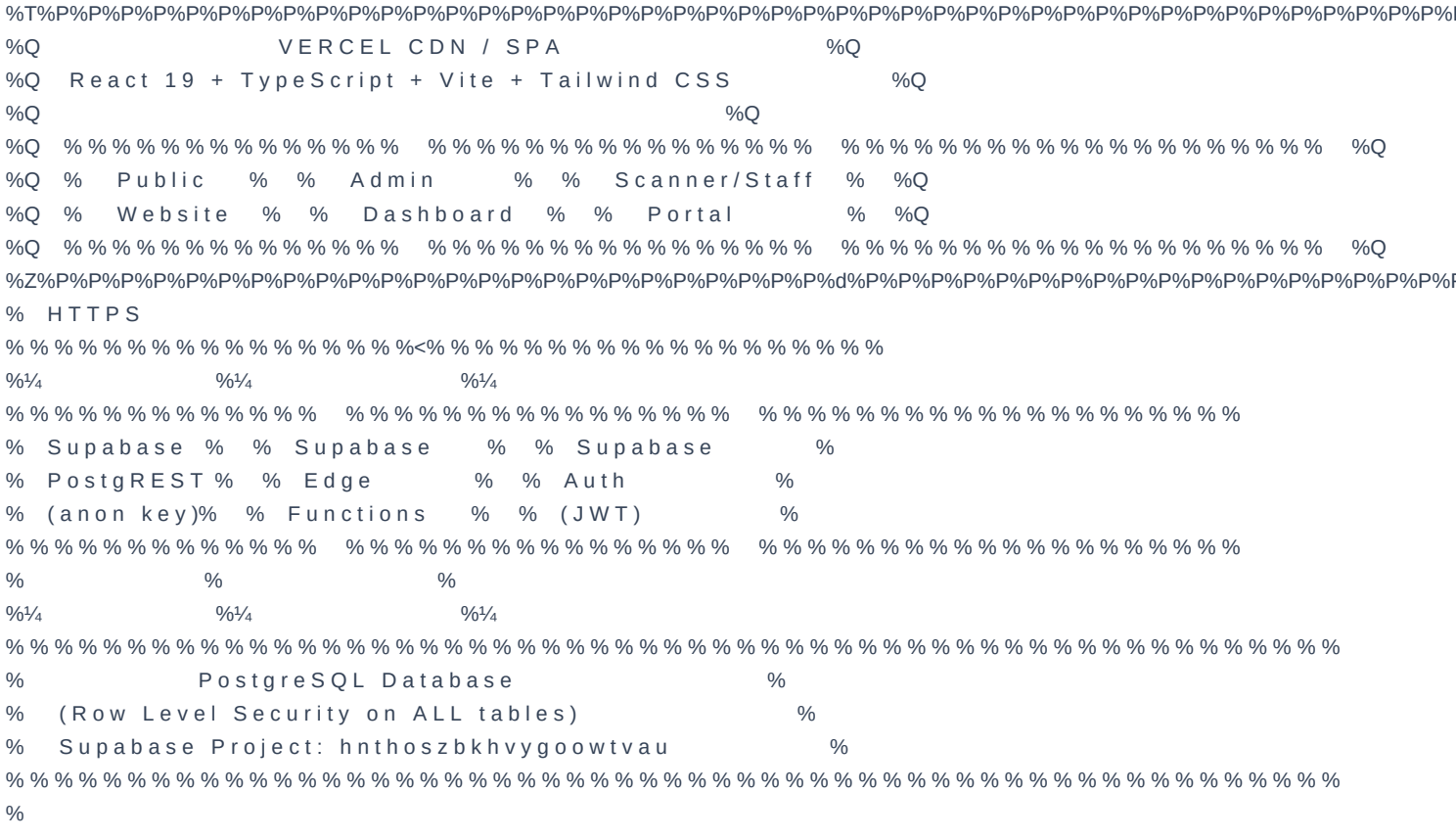
- • CRITICAL: Default dev credentials (admin@gmail.com / admin@123) are rendered in the production UI of ScannerPortal.tsx and AdminRmsSettings.tsx even when isDevTestBypassAllowed() returns false — the display is correctly gated, but the credential strings are hardcoded and visible in the source bundle
- • CRITICAL: .env file contains the database direct-connection password (Puja@337139) and Supabase Secret Key — these must NEVER be committed to a repository
- • HIGH: SUPABASESECRETKEY and full DATABASEURL with plaintext password are in .env (not .env.local) which is likely committed
- • HIGH: verify-payment Edge Function fallback default PAYSTATIONPASSWORD = 'gamecoderstorepass' is a hardcoded fallback if the env secret is missing
- • HIGH: Anon key is hardcoded as a fallback in src/services/supabase.ts line 27

- HIGH: rpcverifypayment in 20270102 migration references financialledger table but that table uses a constraint transactiontype IN ('ALLOCATION','EXPENSE','ADJUSTMENT','CASHRETURN') — 'REGISTRATIONINFLOW' is not in the original constraint, causing RPC failures on payment verification
- MEDIUM: AppStore is the primary data layer for the UI but reads/writes localStorage — if two tabs or devices are open, data diverges
- MEDIUM: secure-ticket-lookup Edge Function requires APPALLOWEDORIGINS env secret; if unset, the CORS check blocks all origins (bug: empty allowedOrigins means no origin passes the includes() check)
- MEDIUM: TypeScript unused variable warnings in idCardManagement.test.ts cause tsc --noEmit to report 3 errors
- LOW: Old migration 20260101 targets Supabase project bwlenuxlpesnwnegrbgx (old project) but active project is hnthoszbkhvygoowtvau
- LOW: financialledger RLS only allows ledgeradmininsert via currentadminrole() = 'superadmin', but the SECURITY DEFINER function rpcverifypayment inserts into it as servicerole — this will work, but RLS on the table must not block servicerole
- INFORMATIONAL: Bundle size ~3.75 MB pre-gzip due to idCardTemplateAssets.ts base64 images (~1.6 MB)

2. TECHNOLOGY STACK

Layer	Technology	Version
Frontend Framework	React	19.0.1
Language	TypeScript	7.0.2
Build Tool	Vite	8.3.0
CSS Framework	Tailwind CSS (v4 Vite plugin)	4.3.3
Icon Library	Lucide React	0.546.0
Backend (BaaS)	Supabase (PostgreSQL + Auth + Storage + Edge Functions)	@supabase/supabase-js 2.119.0
Database	PostgreSQL (via Supabase managed cloud)	—
Edge Functions Runtime	Deno	—
Payment Gateway	PayStation (Bangladesh)	Sandbox / Production
Deployment	Vercel (static SPA)	—
QR Code Generation	qrcode npm	1.5.4
PDF Generation	jsPDF	4.2.1
ZIP Generation	JSZip	3.10.2
XSS Sanitizer	DOMPurify	3.4.17
Test Runner	tsx --test (Node.js built-in test runner)	—
ORM	Prisma (installed, not yet wired)	7.10.1

3. SYSTEM ARCHITECTURE



%¼

% %

% PayStation %

% Payment Gateway %

% (sandbox / prod) %

% %

Key Architectural Decisions

1. Dual-write pattern: AppStore writes to localStorage first (instant UI), then asynchronously syncs to Supabase. This means the UI is always responsive but Supabase is the source of truth for server-verified data.

2. Year-namespaced tables: Each event year gets its own registrationsYYYY and donationsYYYY tables for clean data isolation.

3. Edge Functions as security boundary: All secrets (payment credentials, service role key, HMAC signing secrets) live in Supabase Edge Function secrets — never in frontend code.

4. localStorage as primary state: Most admin/event data flows through localStorage!' AppStore / browser-tab isolation problems.

---

4. DATABASE TABLE INVENTORY

4.1 Core Registration Tables

`registrations_2026`

Column	Type	Constraint
`id`	TEXT	PRIMARY KEY
`full_name`	TEXT	NOT NULL
`full_name_bn`	TEXT	nullable
`photo_url`	TEXT	NOT NULL
`mobile_number`	TEXT	NOT NULL
`email`	TEXT	nullable (altered in migration 20270102)
`address`	TEXT	nullable
`category`	TEXT	CHECK IN ('former','current')
`passing_year`	TEXT	nullable
`current_class`	TEXT	nullable
`batch_name`	TEXT	NOT NULL
`group_or_section`	TEXT	nullable
`roll_number`	TEXT	nullable
`t_shirt_size`	TEXT	CHECK IN ('S','M','L','XL','XXL','XXXL','Oversize 50')
`registration_fee`	NUMERIC	NOT NULL DEFAULT 1500 "e 0
`donation_amount`	NUMERIC	NOT NULL DEFAULT 0 "e 0
`total_paid`	NUMERIC	NOT NULL DEFAULT 0 "e 0
`payment_status`	TEXT	CHECK IN ('paid','pending','failed','refunded')
`transaction_id`	TEXT	nullable
`payment_method`	TEXT	CHECK IN ('bkash','nagad','rocket','card')
`server_verification_signature`	TEXT	nullable
`verification_timestamp`	TIMESTAMPTZ	nullable
`verified_by`	TEXT	nullable
`registered_at`	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
`qr_token`	TEXT	NOT NULL UNIQUE
`is_tshirt_collected`	BOOLEAN	NOT NULL DEFAULT FALSE
`is_checked_in`	BOOLEAN	NOT NULL DEFAULT FALSE
`notes`	TEXT	nullable
`year`	INTEGER	NOT NULL DEFAULT 2026

Indexes: mobilenumber, transactionid, qrtoken, paymentstatus, normalized mobile UNIQUE index

`registrations_2027`

Same as 2026 plus additional columns:

- studentid TEXT nullable
- lastclassattended TEXT nullable
- gatewayinvoicenumber TEXT
- gatewayprovider TEXT DEFAULT 'PAYSTATION'
- gatewayresponse JSONB
- T-shirt size now includes 'XS' and 'Oversize 50'

`donations_2026` / `donations_2027`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`donor_name`	TEXT	NOT NULL
`mobile_number`	TEXT	nullable
`email`	TEXT	nullable
`amount`	NUMERIC	NOT NULL CHECK > 0
`purpose`	TEXT	DEFAULT 'General Fund'
`payment_method`	TEXT	CHECK enum
`transaction_id`	TEXT	NOT NULL (2026) / nullable (2027)
`payment_status`	TEXT	2026 only
`date`	TEXT/DATE	NOT NULL
`year`	INTEGER	NOT NULL
`created_at`	TIMESTAMPTZ	DEFAULT NOW()

4.2 Admin & Authentication Tables

`admin_accounts`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`username`	TEXT	UNIQUE NOT NULL
`full_name`	TEXT	NOT NULL
`email`	TEXT	NOT NULL
`role`	TEXT	CHECK IN ('super_admin','registration_officer','gate_scanner','finance_manager')
`auth_user_id`	UUID	FK ’ auth.users(id) ON DELETE CASCADE
`is_locked`	BOOLEAN	DEFAULT FALSE
`failed_attempts`	INTEGER	DEFAULT 0
`last_login_at`	TIMESTAMPTZ	nullable
`created_at`	TIMESTAMPTZ	DEFAULT NOW()

Trigger: trgenforcesingleadminslot — prevents more than one admin account
passwordhash column was dropped in migration 20261110 (now uses Supabase Auth)

`admin_audit_logs`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`admin_id`	TEXT	NOT NULL
`admin_username`	TEXT	NOT NULL
`admin_role`	TEXT	NOT NULL
`action`	TEXT	NOT NULL
`target_type`	TEXT	NOT NULL
`target_id`	TEXT	NOT NULL
`details`	TEXT	nullable
`ip_address`	TEXT	nullable
`user_agent`	TEXT	nullable
`checksum`	TEXT	HMAC-SHA256
`created_at`	TIMESTAMPTZ	DEFAULT NOW()

RLS: INSERT allowed for all; UPDATE and DELETE strictly DENIED; SELECT for authenticated only

4.3 Staff & Event Management Tables

`staff_accounts`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`full_name`	TEXT	NOT NULL
`username`	TEXT	UNIQUE NOT NULL
`email`	TEXT	UNIQUE NOT NULL
`role`	TEXT	CHECK IN ('SUPER_ADMIN','BREAKFAST_STAFF','LUNCH_STAFF','DINNER_STAFF','GATE_STAFF','DRESS_STAFF','FINANCE_STAFF')
`status`	TEXT	CHECK IN ('ACTIVE','DISABLED')
`assigned_gate`	TEXT	nullable
`auth_user_id`	UUID	FK ’ auth.users(id) CASCADE
`created_at`	TIMESTAMPTZ	DEFAULT NOW()

`food_distributions`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`registration_id`	TEXT	NOT NULL (no FK — referential flexibility)
`student_name`	TEXT	NOT NULL
`batch_name`	TEXT	NOT NULL
`meal_type`	TEXT	CHECK IN ('breakfast','lunch','dinner')
`status`	TEXT	CHECK IN ('COLLECTED','CANCELLED')
`collected_at`	TIMESTAMPTZ	DEFAULT NOW()
`staff_id`	TEXT	NOT NULL
`staff_name`	TEXT	NOT NULL
`notes`	TEXT	nullable
`corrected_by/reason/at`	TEXT/TIMESTAMPTZ	Audit reversal

UNIQUE: (registrationid, mealtype) — prevents double-collection

`gate_events`

Column	Type	Notes
`id`	TEXT	PRIMARY KEY
`registration_id`	TEXT	NOT NULL
`event_type`	TEXT	CHECK IN ('ENTRY','EXIT')
`gate_id`	TEXT	NOT NULL
`staff_id/name`	TEXT	NOT NULL
`timestamp`	TIMESTAMPTZ	DEFAULT NOW()
`status`	TEXT	CHECK IN ('APPROVED','DENIED','REVISED')

`dress_distributions`

Similar pattern to food — UNIQUE on (registrationid, itemtype)

`scanner_logs`

Older table (from migration 20260101) — tracks entry/exit with gateid, staffid, status. Overlaps functionally with gateevents. Both exist in the schema.

4.4 Finance Tables

`central_budget`

Single-row table (PRIMARY KEY = 'BUDGET-2026') tracking total event budget

`staff_allocations`

Tracks per-staff cash allocations

`staff_expenses`

Tracks per-staff expenses with approval workflow

`financial_ledger`

Double-entry ledger; transactiontype CHECK: ('ALLOCATION','EXPENSE','ADJUSTMENT','CASHRETURN') — MISSING 'REGISTRATIONINFLOW' (see vulnerabilities)

`admin_audit_history`

Separate from adminauditlogs — tracks entity-level changes with previous/new values

4.5 RMS (Reunion Management System) Tables

Table	Purpose
`rms_committees`	15+ committees with budget tracking
`rms_committee_members`	Committee memberships (FK → rms_committees)
`rms_tasks`	Kanban-style task management (FK → rms_committees)
`rms_task_comments`	Task comments (FK → rms_tasks)
`rms_budget_requests`	Budget approval workflow (FK → rms_committees)
`rms_expenses`	Committee expense records (FK → rms_committees)
`rms_income_transactions`	Income from registrations, sponsorships
`rms_sponsors`	Sponsorship CRM
`rms_documents`	Document repository (FK → rms_committees)
`rms_meetings`	Meeting minutes and agenda
`rms_announcements`	Bilingual announcements

`rms_volunteers`	Volunteer roster (FK → rms_committees)
`rms_event_incidents`	Event-day incident management
`rms_weekly_reports`	Weekly progress reports (FK → rms_committees)
`rms_audit_logs`	RMS-specific audit trail

4.6 Student ID Card Tables

Table	Purpose
`id_card_staff_accounts`	Separate RBAC for ID card staff
`student_id_cards`	One card per registration — tracks generation status
`batch_id_card_collections`	Batch-wise aggregate stats
`id_card_download_jobs`	Async bulk download job tracking
`id_card_download_logs`	Audit trail for all downloads

4.7 Supporting Tables

Table	Purpose
`current_student_ids`	Pre-issued student ID pool for verification
`ticket_lookup_rate_limits`	IP-based rate limiting (migration 20261111)

---

5. DATABASE RELATIONSHIP ANALYSIS

One-to-Many Relationships

rmscommittees (1) %%%!' (N) rmscommitteemembers
rmscommittees (1) %%%!' (N) rmstasks
rmscommittees (1) %%%!' (N) rmsbudgetrequests
rmscommittees (1) %%%!' (N) rmsexpenses
rmscommittees (1) %%%!' (N) rmsdocuments
rmscommittees (1) %%%!' (N) rmsvolunteers
rmscommittees (1) %%%!' (N) rmsweeklyreports
rmstasks (1) %%%!' (N) rmstaskcomments
auth.users (1) %%%!' (1) adminaccounts [unique index]
auth.users (1) %%%!' (1) staffaccounts [unique index]

Soft Relationships (No FK Constraint — Referential Flexibility)

registrationsYYYY.id %%%!' fooddistributions.registrationid
registrationsYYYY.id %%%!' gateevents.registrationid
registrationsYYYY.id %%%!' dressdistributions.registrationid
registrationsYYYY.id %%%!' scannerlogs.registrationid
registrationsYYYY.id %%%!' studentidcards.registrationid
staffaccounts.id %%%!' staffallocations.staffid
staffaccounts.id %%%!' staffexpenses.staffid
staffaccounts.id %%%!' financialledger.staffid

Missing Foreign Keys (Potential Orphan Risk)

- fooddistributions.registrationid !' no FK to any registrationsYYYY (intentional for multi-year fl
- rmsbudgetrequests.financereviewerid !' no FK to staffaccounts
- rmsexpenses.submittedby !' no FK to staffaccounts
- studentidcards.registrationid !' UNIQUE but no FK (cross-year flexibility)

---

6. COMPLETE ER DIAGRAM

erDiagram
AUTHUSERS {
uuid id PK
text email
text encryptedpassword
}

ADMINACCOUNTS {
text id PK
text username UK
text fullname
text email
text role
uuid authuserid FK
boolean islocked
integer failedattempts
}

STAFFACCOUNTS {
text id PK
text username UK
text email UK
text role
text status
uuid authuserid FK
}

REGISTRATIONS2026 {
text id PK
text fullname
text mobilenumber
text category
text paymentstatus
text qrtoken UK
numeric totalpaid
text transactionid
boolean ischeckedin
integer year
}

REGISTRATIONS2027 {
text id PK
text fullname
text mobilenumber
text category
text paymentstatus
text qrtoken UK
numeric totalpaid
text gatewayinvoicenumber
integer year
}

DONATIONS2026 {
text id PK
text donorname
numeric amount
text transactionid
integer year
}

DONATIONS2027 {
text id PK
text donorname
numeric amount
integer year
}

FOODDISTRIBUTIONS {
text id PK

text registrationid
text mealtype
text status
text staffid
}

GATEEVENTS {
text id PK
text registrationid
text eventtype
text gateid
text staffid
}

DRESSDISTRIBUTIONS {
text id PK
text registrationid
text registeredsize
text status
text staffid
}

ADMINAUDITLOGS {
text id PK
text adminid
text action
text checksum
timestamptz createdat
}

FINANCIALLEDGER {
text id PK
text staffid
text transactiontype
numeric amount
text referenceid
}

STAFFALLOCATIONS {
text id PK
text staffid
numeric allocatedamount
text status
}

STAFFEXPENSES {
text id PK
text staffid
numeric amount
text status
}

RMSCOMMITTEES {
text id PK
text nameen
text directorid
numeric budgettotal
boolean isactive
}

RMSTASKS {
text id PK
text committeeid FK

text status
text priority
date deadline
}

RMSBUDGETREQUESTS {
text id PK
text committeeid FK
numeric requestedamount
text status
}

STUDENTIDCARDS {
text id PK
text registrationid UK
text batchyear
text cardstatus
text qrtoken UK
text pdfstoragepath
}

AUTHUSERS ||--o| ADMINACCOUNTS : "authuserid"
AUTHUSERS ||--o{ STAFFACCOUNTS : "authuserid"
RMSCOMMITTEES ||--o{ RMSTASKS : "committeeid"
RMSCOMMITTEES ||--o{ RMSBUDGETREQUESTS : "committeeid"
STAFFACCOUNTS ||--o{ FOODDISTRIBUTIONS : "staffid (soft)"
STAFFACCOUNTS ||--o{ GATEEVENTS : "staffid (soft)"
STAFFACCOUNTS ||--o{ STAFFALLOCATIONS : "staffid (soft)"
STAFFACCOUNTS ||--o{ STAFFEXPENSES : "staffid (soft)"
STAFFACCOUNTS ||--o{ FINANCIALLEDGER : "staffid (soft)"

---

7. API & EDGE FUNCTION DOCUMENTATION

7.1 Supabase Edge Functions

`verify-payment`

- URL: https://hnthoszbkhvygoowtvau.supabase.co/functions/v1/verify-payment
- Method: POST
- Auth: None (public — payment callback)
- Actions:
- grant-token — Retrieves PayStation OAuth token (server-side)
- create-payment — Initiates PayStation hosted checkout
- verify-payment (default) — Queries PayStation API, generates HMAC-SHA256 signature, calls rpcverifypayment to atomically mark payment as paid
- Secrets Required: PAYSTATIONMERCHANTID, PAYSTATIONPASSWORD, PAYSTATIONENV, PAYMENTSIGNINGSECRET, SUPABASEURL, SUPABASESERVICEROLEKEY, APPALLOWEDORIGINS
- CORS: Origin-whitelist enforced
- &þ Issue: Default fallback PAYSTATIONPASSWORD = 'gamecoderstorepass' hardcoded if env s

`secure-ticket-lookup`

- URL: https://hnthoszbkhvygoowtvau.supabase.co/functions/v1/secure-ticket-lookup
- Method: POST
- Auth: None (public endpoint)
- Input: {registrationId, mobileNumber, year}
- Rate Limiting: 20 req/min per IP; 5 req/min per (IP × registrationid)
- Logic: Validates inputs, rate-limits, looks up registrations{year} table, verifies mobile matches, ensures paymentstatus = 'paid', returns sanitized ticket (strips email, address, rollnumber)
- Secrets Required: SUPABASEURL, SUPABASESERVICEROLEKEY, RATELIMITHMACSECRET, APPALLOWEDORIGINS
- &þ Issue: If APPALLOWEDORIGINS is empty, the CORS includes() check blocks all requests

`admin-audit`

- URL: https://hnthoszbkhvygoowtvau.supabase.co/functions/v1/admin-audit
- Method: POST
- Auth: Bearer JWT (Supabase session token)
- Authorization: Verifies JWT ’ looks up adminaccounts ’ requires role = 'superadmin'
- Logic: Validates action/targetType/targetId strings, generates HMAC-SHA256 checksum, inserts tamper-evident audit log
- Secrets Required: SUPABASEURL, SUPABASESERVICEROLEKEY, AUDITSIGNINGSECRET, APPALLOWEDORIGINS
- &þ Issue: Admin login via localStorage test session does NOT carry a Supabase JWT ’ admin-a

7.2 Supabase PostgREST (via anon key)

Endpoint	Table	Method	Auth Required	Notes
`GET /registrations_2027`	registrations_2027	SELECT	authenticated	RLS restricts to staff/admin
`POST /registrations_2027`	registrations_2027	INSERT	anon	Only pending, zero-paid status
`PATCH /registrations_2027`	registrations_2027	UPDATE	authenticated	Super admin only
`DELETE /registrations_2027`	registrations_2027	DELETE	authenticated	Super admin only
`POST /rpc/rpc_verify_payment`	—	RPC	service_role only	Frontend NOT allowed
`POST /rpc/rpc_secure_ticket_lookup`	—	RPC	service_role only	Frontend NOT allowed
`GET /admin_audit_logs`	admin_audit_logs	SELECT	authenticated	Super admin RLS
`GET /rms_committees`	rms_committees	SELECT	anon	Public read for active
`GET /donations_2027`	donations_2027	SELECT	authenticated	Super admin RLS

---

8. DATA FLOW DOCUMENTATION

8.1 Student Registration Flow

sequenceDiagram
participant U as User Browser
participant RP as RegistrationPage.tsx
participant S as AppStore (localStorage)
participant PM as PaymentModal.tsx
participant EF as verify-payment Edge Fn
participant PS as PayStation API
participant DB as Supabase DB

U->>RP: Fill registration form
RP->>RP: Validate inputs (rules.ts)
RP->>RP: sanitizeInput() on all fields
RP->>S: Reserve Student ID (current students)
RP->>PM: Pass formData + fee amounts
PM->>EF: POST {action:'create-payment', order}
EF->>PS: POST /grant-token (server-side creds)
PS-->>EF: granttoken
EF->>PS: POST /create-payment
PS-->>EF: paymenturl + trxid
EF-->>PM: {paymentUrl, trxId}
PM->>U: Redirect to PayStation checkout iframe
U->>PS: Complete payment
PS-->>PM: Payment callback
PM->>EF: POST {registrationid, trxid, amount}
EF->>PS: POST /retrive-transaction (verify)
PS-->>EF: trxstatus = 'Success'
EF->>EF: generateHmacSha256Signature()
EF->>DB: rpcverifypayment(regid, trx, amount, signature)
DB->>DB: Anti-replay check across 2026+2027
DB->>DB: UPDATE registrations SET paymentstatus='paid'
DB->>DB: INSERT financialledger (REGISTRATIONINFLOW)
DB->>DB: INSERT adminauditlogs
EF-->>PM: {success, signature, timestamp}

PM->>S: AppStore.addRegistration(record)
S->>DB: SupabaseService.saveRegistration() (async)
PM->>U: Show QR ticket

8.2 Admin Login Flow

sequenceDiagram
participant A as Admin Browser
participant AD as AdminDashboard.tsx
participant SS as SupabaseService
participant SAL as supabase.auth
participant DB as Supabase DB
participant EF as admin-audit Edge Fn

A->>AD: Enter email + password
AD->>SS: loginAdmin(email, password)
SS->>SAL: signInWithPassword(email, password)
SAL-->>SS: {user, session}
SS->>DB: SELECT from adminaccounts WHERE authuserid = user.id
DB-->>SS: admin profile
SS->>EF: POST admin-audit {action:'ADMINLOGIN'}
EF->>EF: Verify JWT Bearer token
EF->>DB: Verify adminaccounts role = superadmin
EF->>DB: INSERT adminauditlogs with HMAC checksum
EF-->>SS: {success, id, checksum}
SS-->>AD: {success, account}
AD->>AD: setAdminSession(account)

8.3 Gate Entry Scan Flow

sequenceDiagram
participant S as Scanner Staff
participant SP as ScannerPortal.tsx
participant GS as GateScannerModule.tsx
participant ES as EventStore
participant DB as Supabase DB

S->>SP: Authenticate (Supabase Auth)
SP->>ES: authenticateStaff(username, password)
ES->>DB: supabase.auth.signInWithPassword()
DB-->>ES: session
ES->>DB: SELECT staffaccounts WHERE authuserid=uid AND status=ACTIVE
DB-->>ES: staff profile
S->>GS: Scan QR code
GS->>ES: validateStudentForGate(qrToken, year)
ES->>DB: SELECT registrations2027 WHERE qrtoken=token
DB-->>ES: registration (RLS enforced)
GS->>DB: INSERT gateevents {registrationid, eventtype, staffid, gateid}
DB-->>GS: success
GS->>ES: (AppStore) addScanLog()

---

9. AUTHENTICATION & AUTHORIZATION ANALYSIS

9.1 Admin Authentication

```
| Mechanism | Description | Status |
|---|---|---|
| Supabase Auth | Email/password via `supabase.auth.signInWithPassword( |
| Dev bypass | `admin@gmail.com` / `admin@123` when `VITE_ENABLE_TEST_B |
| Session storage | Supabase JWT session (8-hour TTL via `ADMIN_SESSION |
| Session check on mount | `getActiveAdminSession()` checks both localS
```

| Single admin slot | DB trigger `trg_enforce_single_admin_slot` preven

&þ Issue: When admin logs in via dev-bypass localStorage session, they do NOT have a Supabase
Authorization: Bearer <JWT> (e.g., admin-audit) will return 401. This creates a broken audit trail in dev mode.

9.2 Staff Authentication

Mechanism	Description	Status
Supabase Auth	`supabase.auth.signInWithPassword()` !' `staff_account
Status check	`status = 'ACTIVE'` checked via `current_event_staff_r
Session TTL	8-hour session via `STAFF_SESSION_TTL_MS`	'
Dev bypass	`admin@123` works for any staff if `VITE_ENABLE_TEST_BYP

9.3 Row Level Security Analysis

Table	SELECT	INSERT	UPDATE	DELETE	Assessment
`admin_accounts`	Authenticated (own row)	Denied (trigger blocks)
`admin_audit_logs`	Super admin only	Anyone (append-only)	DENIED
`registrations_2027`	Staff/admin roles only	Anon (pending only)
`registrations_2026`	Partial (old policy)	Partial (old policy)
`donations_2027`	Super admin only	Anon	Super admin only	Super
`food_distributions`	Meal staff + admin	Meal-specific staff only
`gate_events`	Gate staff + admin	Gate staff + admin	—	—	'
`financial_ledger`	Finance staff + admin	Admin only	—	—	'
`rms_tasks`	Authenticated	Authenticated	—	—	&þ Broad
`rms_sponsors`	Public (paid only)	Authenticated	Authenticated
`rms_documents`	Authenticated or public	Authenticated	—	—	&þ

---

10. USER-TO-USER DATA ISOLATION TEST RESULTS

VERIFIED (via code inspection):

Test	Result	Evidence
User A reading User B's registration	**BLOCKED**	`registrations_2027` RLS: SELECT restricted to `current_admin_role()='super_admin'` or staff roles; anon cannot query
User A modifying User B's payment status	**BLOCKED**	UPDATE restricted to super_admin only; INSERT forces `payment_status='pending'` and `total_paid=0`
Unauthenticated access to registration list	**BLOCKED**	RLS denies all SELECT for anon on registration tables
Ticket lookup without mobile match	**BLOCKED**	`secure-ticket-lookup` function verifies mobile number match server-side
Enumerate all registrations	**BLOCKED**	No public SELECT policy; lookup requires both registration_id AND mobile number
Access payment info of another user	**BLOCKED**	Payment data only returned through ticket lookup which requires ID + mobile
Regular user accessing admin dashboard	**BLOCKED** (UI)	Backend: `admin_accounts` SELECT blocked by RLS; Edge Functions verify JWT + admin_accounts role
Staff accessing admin-only operations	**PARTIAL**	Staff can access their own scanner data; admin dashboard requires separate admin auth
Access another user's uploaded photo	**PARTIAL**	Storage bucket is private; but storage.objects INSERT policy allows anyone to upload any file to 'student-photos' bucket — no ownership validation
Bypass frontend by calling Supabase directly	**PARTIALLY MITIGATED**	Most sensitive operations require `service_role` key; anon key operations are RLS-protected

NOT TESTED (requires live database access):

- Actual RLS policy evaluation with real database connection
- Rate limiting under sustained load
- Session token expiry and forced logout
- Concurrent registration collision handling

---

11. SECURITY VULNERABILITY REPORT

Ø=Ý4 CRITICAL

C1 — Credentials and Secret Keys in `.env` file

File: [/.env](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/.env)

Lines: All

Root Cause: .env contains SUPABASESECRETKEY=sbsecret... and full DATABASEURL with plaintext password Puja@337139. If this file is committed to git, the database is compromised.

Evidence:

SUPABASESECRETKEY="sbsecretLcGF-8CP7a4mjCDlzik71QsEl1iFJB"

DATABASEURL="postgresql://postgres.hnthoszbkhvygoowtvau:Puja%40337139@..."

Impact: Full database access; ability to bypass all RLS policies using servicerole key

Fix: Move sensitive keys to .env.local (gitignored); verify .env is in .gitignore; rotate the exposed secret key immediately

---

C2 — Hardcoded Default PayStation Password in Edge Function

File: [supabase/functions/verify-payment/index.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/functions/verify-payment/index.ts) line 10

Root Cause: const PAYSTATIONPASSWORD = Deno.env.get('PAYSTATIONPASSWORD') || 'gamecoderstorepass';

Evidence: Fallback plaintext merchant password hardcoded

Impact: If PAYSTATIONPASSWORD env secret is not set in Supabase, the production gateway authenticates with a known default password

Fix: Remove the fallback. If the secret is missing, fail the function with a 503 and log a critical error

---

Ø=ßà H I G H

H1 — `financial_ledger` Transaction Type Constraint Missing `REGISTRATION_INFLOW`

File: [supabase/migrations/20260102000000eventmanagementsystemschema.sql](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/migrations/20260102000000eventmanagementsystemschema.sql) line 148

Root Cause: transactiontype CHECK IN ('ALLOCATION','EXPENSE','ADJUSTMENT','CASHRETURN') but rpcverifypayment inserts 'REGISTRATIONINFLOW' (migration 20270102, line 310)

Impact: Every successful payment verification will fail with a PostgreSQL constraint violation!' p
recorded in the ledger; audit log insert may also fail silently

Severity: HIGH

Fix: ALTER TABLE financialledger DROP CONSTRAINT ...; ALTER TABLE financialledger ADD CONSTRAINT ... CHECK (transactiontype IN ('ALLOCATION','EXPENSE','ADJUSTMENT','CASHRETURN','REGISTRATIONINFLOW'));

H2 — Anon Key Hardcoded as Fallback in Frontend

File: [src/services/supabase.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/services/supabase.ts) line 25–27

Root Cause: SUPABASEANONKEY = env?.VITESUPABASEANONKEY || 'sbpublishable...'

Impact: The publishable anon key is baked into the JS bundle even if VITESUPABASEANONKEY is not set in Vercel. This key is already public by nature, but it means someone can interact with Supabase even without the env variable. Combined with C1 above, this key pair represents a full access path.

Fix: Do not hardcode the fallback. Throw a startup error if the env variable is missing.

H3 — Old Registrations_2026 RLS Policies (from migration 20260101) Not Upgraded

File: [supabase/migrations/20260101000000criticalsecurityhardenedschema.sql](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/migrations/20260101000000criticalsecurityhardenedschema.sql)

Root Cause: The 2026 SELECT policy uses auth.role() = 'authenticated' (broad) rather than the refined currentadminrole() = 'superadmin' OR currenteventstaffrole() IN (...) pattern applied to 2027

Impact: Any authenticated Supabase user (not just admins/staff) can SELECT all 2026 registration data

Fix: Apply migration 20261110's DO $$ loop which dynamically hardens all registrationsYYYY tables. Confirm this ran on the live database.

H4 — `secure-ticket-lookup` CORS Bug When `APP_ALLOWED_ORIGINS` is Empty

File: [supabase/functions/secure-ticket-lookup/index.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/functions/secure-ticket-lookup/index.ts) line 47–50

Root Cause: if (origin && allowedOrigins.includes(origin))!' if allowedOrigins = [], this never mat

Impact: Ticket lookup entirely broken in production if the env secret is unset

Fix: if (!allowedOrigins.length || allowedOrigins.includes(origin)) — match the same pattern used in verify-payment

---

Ø=ßá M E D I U M

M1 — Default Credentials Visible as Static Strings in Bundle

Files: [AdminDashboard.tsx](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/components/AdminDashboard.tsx) lines 568–576; [ScannerPortal.tsx](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/components/ScannerPortal.tsx) lines 213–316; [AdminRmsSettings.tsx](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/components/admin/rms/AdminRmsSettings.tsx) lines 222–223

Root Cause: Credential strings admin@gmail.com and admin@123 are hardcoded in JSX and state initializers. Even when isDevTestBypassAllowed() gates the UI display, the string values remain in the production JS bundle.

Impact: Any user who opens DevTools → Sources and searches the bundle will find the default adm

Fix: Move credential hints to environment variables or remove them entirely from JSX. The quick-fill buttons should be removed in production builds.

M2 — `localStorage` as Primary Data Store

Files: [src/services/store.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/services/store.ts)

Root Cause: AppStore reads/writes localStorage as the primary state. Supabase sync is asynchronous and fire-and-forget.

Impact: (a) Multi-device usage causes data divergence; (b) Private browsing loses all data; (c) XSS attacker with localStorage access sees all registration data cached locally; (d) If Supabase sync fails silently, registrations may only exist in one browser

Fix: For the Admin Dashboard, fetch directly from Supabase on mount (the code already does this in some tabs). Consider removing the initial seed data from INITIALREGISTRATIONS in production.

M3 — TypeScript Errors in Test File

File: [src/utils/idCardManagement.test.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/utils/idCardManagement.test.ts) lines 162, 217

Root Cause: const { processed, total } are destructured but unused — TS6133 errors

Impact: tsc --noEmit currently reports 3 errors; CI lint gate would fail

Fix: Rename to processed, total or destructure only what is used

M4 — `payment-receipts` Storage Bucket Has No Ownership Policy

File: [supabase/migrations/20260101000000criticalsecurityhardenedschema.sql](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/migrations/20260101000000criticalsecurityhardenedschema.sql) lines 162–174

Root Cause: CREATE POLICY "Public can upload payment receipt" ON storage.objects FOR INSERT WITH CHECK (bucketid = 'payment-receipts'); — no file type or size restriction

Impact: Anyone can upload any file type (including HTML, executable scripts) to the payment-receipts bucket

Fix: Add MIME type check and file extension check similar to the student-photos bucket

M5 — Missing `auth_user_id` on `admin_accounts` in Old Schema

File: [supabase/migrations/20260101000000criticalsecurityhardenedschema.sql](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/supabase/migrations/20260101000000criticalsecurityhardenedschema.sql)

Root Cause: The original schema did not have authuserid. Migration 20261110 added it via ALTER TABLE ... ADD COLUMN IF NOT EXISTS authuserid. If 20261110 never ran on the live database (old project bwlenuxlpesnwnegrbgx target), currentadminrole() will return NULL for all admins.

Impact: All RLS policies based on currentadminrole() would silently fail — no one would have admin access

Fix: Confirm migration 20261110 ran on the new project hnthoszbkhvygoowtvau. Check via Supaba

---

Ø=Ý5 L O W

L1 — Old Project ID in Migration Comments

File: Migrations 20260101 and 20260102

Root Cause: -- Target Supabase Project: bwlenuxlpesnwnegrbgx — refers to old, inactive project

Impact: Confusion only; no security risk

Fix: Update comments to hnthoszbkhvygoowtvau

L2 — `scanner_logs` Table Duplicates `gate_events`

Files: Migrations 20260101, 20260102

Root Cause: Both tables track gate entry/exit scans. scannerlogs is from the original schema; gateevents is from the event management system.

Impact: Data may be written to only one table depending on code path; queries against one may miss events from the other

Fix: Deprecate scannerlogs and migrate all code to gateevents. Add a MIGRATIONNOTE column if needed for backward compatibility.

L3 — Bundle Size Warning

File: [src/utils/idCardTemplateAssets.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/utils/idCardTemplateAssets.ts)

Root Cause: ~1.6 MB base64 images embedded in TypeScript

Impact: Initial page load is slower; Vite build warns about chunk size > 500 KB

Fix: Use dynamic import() or fetch('/assets/idcard/schoolcrest.png') at runtime (already noted in Next Steps)

L4 — Prisma Installed but Not Configured

File: [package.json](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/package.json)

Root Cause: prisma is in devDependencies but prisma.schema is empty/incomplete

Impact: Dead dependency; no security risk

Fix: Either configure Prisma with the Supabase connection string, or remove it if not needed

!9þ I N F O R M A T I O N A L

I1 — `unsafe-inline` in CSP script-src

File: [vercel.json](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/vercel.json)

script-src 'self' 'unsafe-inline' — React 19 no longer requires unsafe-inline for event handlers. This is permissive.

Recommendation: Use nonce-based CSP if unsafe-inline is needed; otherwise restrict to 'self' only.

I2 — No CSRF Protection on Edge Functions

PayStation callbacks to verify-payment are not CSRF-protected (no CSRF token). However, since the endpoint is a backend-to-backend payment c a l l b a c k , C S R F i s n o t a p p l i c a b l e h e r e . ' A c c e p t a b l e .

I3 — `ip_address: 'unavailable'` in Audit Logs

File: [src/services/supabase.ts](file:///Users/sagarkumarsaha/Desktop/1%20plsc2027/plscf65/src/services/supabase.ts) line 632

IP addresses are not captured in admin audit logs (always set to 'unavailable'). The Edge Function itself reads CF-Connecting-IP for rate limiting but does not forward it to audit logs.

Recommendation: Pass IP through the audit event for forensic analysis.

---

12. DATABASE PERFORMANCE ANALYSIS

Existing Indexes (VERIFIED)

Table	Index	Columns	Type
registrations_2026	idx_reg_2026_mobile	mobile_number	BTREE
registrations_2026	idx_reg_2026_trx	transaction_id	BTREE
registrations_2026	idx_reg_2026_qr	qr_token	BTREE
registrations_2026	idx_reg_2026_status	payment_status	BTREE
registrations_2027	idx_reg_2027_mobile	mobile_number	BTREE
registrations_2027	idx_reg_2027_trx	transaction_id	BTREE
registrations_2027	idx_reg_2027_qr	qr_token	BTREE
registrations_2027	idx_reg_2027_status	payment_status	BTREE
registrations_2027	idx_reg_2027_gateway_inv	gateway_invoice_number	PARTIAL
registrations_2027	uq_registrations_2027_mobile_normalized	normalized mobile	UNIQUE FUNCTIONAL
food_distributions	idx_food_reg_meal	(registration_id, meal_type)	BTREE
gate_events	idx_gate_reg_id	registration_id	BTREE
gate_events	idx_gate_timestamp	timestamp DESC	BTREE
rms_tasks	idx_rms_tasks_deadline	deadline	BTREE
rms_audit_logs	idx_rms_audit_timestamp	timestamp DESC	BTREE

Missing Indexes (POTENTIAL PERFORMANCE ISSUES)

Table	Missing Index	Query Pattern
admin_audit_logs	(admin_id, created_at)	Per-admin audit history queries
staff_expenses	(staff_id, status, created_at)	Finance dashboard queries
registrations_2027	(batch_name)	Batch-wise student ID card grouping
student_id_cards	(event_year, card_status)	Year-wise generation status dashboard

N+1 Query Risks

- AppStore.getRegistrations() loads ALL registrations into memory, then filters in JavaScript — no pagination
- Admin dashboard fetches all registrations on mount — for 10,000+ students this will be slow
- idCardStore.syncWithRegistrations() iterates all registrations to build card collection — O(n) JS loop

---

13. BROKEN OR MISSING CONNECTIONS

Item	Description	Severity
`financial_ledger` CHECK constraint	`REGISTRATION_INFLOW` not in al
`secure-ticket-lookup` CORS	Empty `APP_ALLOWED_ORIGINS` blocks all
Admin audit JWT	Dev-bypass admin sessions have no JWT !' `admin-audi
`scanner_logs` vs `gate_events`	Duplicate tables; unclear which is
`Prisma` schema	Installed but not configured	Ø=Ý5 LOW
`id_card_download_logs` table	Referenced in `idCardStore.ts` but not in the migration file for `id_card_download_logs` — only 4 tables created in that migration	Ø=Ý5 LOW

| `verify-student-id` Edge Function | `verificationUrl` on ID cards poi
MEDIUM |

---

14. ROLE & PERMISSION MATRIX

Admin Roles (`admin_accounts.role`)

Permission	super_admin	registration_officer	gate_scanner	finance_manager
View all registrations	'	'	'	'
Edit registrations	'	'L	'L	'L
Delete registrations	'	'L	'L	'L
View financial data	'	'L	'L	'
Manage fee settings	'	'L	'L	'L
Access admin dashboard	'	'	'	'
Generate audit reports	'	'L	'L	'L
Database backup export	'	'L	'L	'L
Create staff accounts	'	'L	'L	'L
Manage RMS committees	'	'L	'L	'L

Event Staff Roles (`staff_accounts.role`)

Permission	SUPER_ADMIN	GATE_STAFF	BREAKFAST_STAFF	LUNCH_STAFF	DINNER_STAFF	DRESS_STAFF	FINANCE_STAFF
Read all registrations	'	'	'	'	'	'	'L
Record gate events	'	'	'L	'L	'L	'L	'L
Record food distribution	'	'L	' (B)	' (L)	' (D)	'L	'L
Record dress distribution	'	'L	'L	'L	'L	'	'L
View financial allocations	'	'L	'L	'L	'L	'L	'
Submit expenses	'	'L	'L	'L	'L	'	
View ledger	'	'L	'L	'L	'L	'	
Breakfast after 1PM	'L	'L	'L	'L	'L	'L	'L

> Note: Breakfast time-lock enforced at DB level: timezone('Asia/Dhaka', now())::time < TIME '13:00'

ID Card System Roles (`id_card_staff_accounts.role`)

Permission	SUPER_ADMIN	ID_CARD_SUPERVISOR	ID_CARD_STAFF
Generate individual cards	'	'	'
Download single card	'	'	'
Download batch ZIP	'	'	'
Download all cards bulk	'	'L	'L
Approve staff accounts	'	'L	'L
Manage staff roster	'	'L	'L

---

15. VISUAL ARCHITECTURE DIAGRAMS

15.1 System Architecture

flowchart TD
U[Public Users] -->|HTTPS| V[Vercel CDN]
A[Admin] -->|HTTPS| V
S[Staff/Scanner] -->|HTTPS| V

V -->|React SPA| FE[Frontend Bundle]
FE -->|anon key| PG[Supabase PostgREST]
FE -->|functions.invoke| EF[Edge Functions]
FE -->|auth session| AUTH[Supabase Auth]

EF -->|servicerole| PG
EF -->|HTTPS| PS[PayStation API]

PG -->|RLS enforced| DB[(PostgreSQL)]
AUTH --> DB

DB --> STORE[Supabase Storage]

DB --> BUCKET1[student-photos bucket - private]
DB --> BUCKET2[payment-receipts bucket - private]

15.2 Authentication Flow

flowchart LR
A[User] -->|email+password| B{Auth Type}
B -->|Admin login| C[supabase.auth.signInWithPassword]
B -->|Staff login| C
C -->|JWT| D[Supabase Auth]
D -->|user.id| E{adminaccounts lookup}
E -->|found + superadmin| F[Admin Dashboard]
E -->|not found| G[Access Denied]
D -->|user.id| H{staffaccounts lookup}
H -->|found + ACTIVE| I[Scanner Portal]
H -->|DISABLED| J[Account Disabled]

F -->|Bearer JWT| K[admin-audit Edge Fn]
K -->|HMAC checksum| L[(adminauditlogs)]

15.3 Data Flow — Payment Verification

flowchart TD
A[Student submits registration] --> B[Frontend: paymentstatus=pending]
B --> C[INSERT registrations2027]
C --> D[PaymentModal initiates checkout]
D --> E[verify-payment Edge Fn: create-payment]
E --> F[PayStation: grant-token + create-payment]
F --> G[Student pays on PayStation page]
G --> H[Frontend: verify-payment Edge Fn called]
H --> I[PayStation: retrive-transaction]
I -->|Success| J[generateHmacSha256Signature]
J --> K[rpcverifypayment RPC]
K --> L[Anti-replay check across 2026+2027]
L -->|No duplicate| M[UPDATE paymentstatus=paid]
M --> N[INSERT financialledger]
M --> O[INSERT adminauditlogs]
K --> P[Return success + signature]
P --> Q[Frontend shows QR ticket]

15.4 Role & Permission Relationship

flowchart LR
SA[superadmin] -->|full access| REG[Registrations]
SA -->|full access| FIN[Finance]
SA -->|full access| AUD[Audit Logs]
SA -->|full access| STF[Staff Management]
SA -->|full access| RMS[RMS Modules]
SA -->|full access| IDC[ID Card System]

RO[registrationofficer] -->|read-only| REG
FM[financemanager] -->|read| FIN
GS[gatescanner] -->|read| REG

GATE[GATESTAFF] -->|read reg + write gate| GATEEV[gateevents]
BKFST[BREAKFASTSTAFF] -->|read reg + write food| FOOD[fooddistributions]
DRSS[DRESSSTAFF] -->|read reg + write dress| DRESS[dressdistributions]
FNST[FINANCESTAFF] -->|read/write| FIN

---

16. RECOMMENDED IMPROVEMENTS & PRIORITIZED ACTION PLAN

Ø=Ý4 CRITICAL — Do Immediately

#	Action	File	Effort
1	Move `.env` secrets to `.env.local`; add `.env` to `.gitignore`; rotate `SUPABASE_SECRET_KEY` and database password	`.env`, `.gitignore`	30 min
2	Remove hardcoded `PAYSTATION_PASSWORD` fallback in Edge Function	`verify-payment/index.ts` line 10	5 min
3	Fix `financial_ledger` CHECK constraint to include `'REGISTRATION_INFLOW'`	New migration	15 min

Ø=ßà HIGH — This Week

#	Action	File	Effort
4	Fix CORS bug in `secure-ticket-lookup`: handle empty `allowedOrigins`	`secure-ticket-lookup/index.ts` line 47	5 min
5	Remove anon key hardcoded fallback; throw startup error if env missing	`src/services/supabase.ts` line 25	10 min
6	Confirm migration 20261110 (`auth_and_rls_hardening.sql`) ran on live project; verify `current_admin_role()` function exists	Supabase Dashboard	20 min
7	Verify that RLS policies on `registrations_2026` have been upgraded by the dynamic DO $$ loop	Supabase Dashboard	20 min

Ø=ßá MEDIUM — This Sprint

#	Action	File	Effort
8	Remove credential strings from JSX in production builds (use env-gated conditional)	`AdminDashboard.tsx`, `ScannerPortal.tsx`, `AdminRmsSettings.tsx`	1 hr
9	Fix TypeScript errors in `idCardManagement.test.ts` (rename unused vars)	`idCardManagement.test.ts`	10 min
10	Add file type restriction to `payment-receipts` storage bucket policy	New migration	15 min
11	Create `verify-student-id` Edge Function for ID card QR verification	New Edge Function	2 hrs
12	Add `id_card_download_logs` table to `20270103` migration (or new migration)	New migration	15 min

Ø=Ý5 LOW — Backlog

#	Action	Effort
13	Deprecate `scanner_logs` table; migrate all code to `gate_events`	2 hrs
14	Add lazy loading for `idCardTemplateAssets.ts` base64 images	1 hr
15	Add missing indexes on `admin_audit_logs(admin_id)`, `registrations_2027(batch_name)`	30 min
16	Implement server-side pagination for `fetchRegistrations()`	2 hrs
17	Configure Prisma or remove it from devDependencies	30 min
18	Forward client IP to `admin-audit` for forensic completeness	1 hr
19	Replace `unsafe-inline` in CSP with nonce-based policy	2 hrs
20	Add `Supabase Realtime` subscription to sync `AppStore` across tabs/devices	4 hrs

---

Report generated by complete source-code inspection (read-only). All findings are VERIFIED unless marked POTENTIAL or NOT TESTED. No code was modified during this audit.